salesianos
COLEGIO SAN IGNACIO
CÁDIZ

Entornos de Desarrollo

UD5. Diseño y Realización de Pruebas
(2. Procedimientos y Casos de Prueba)

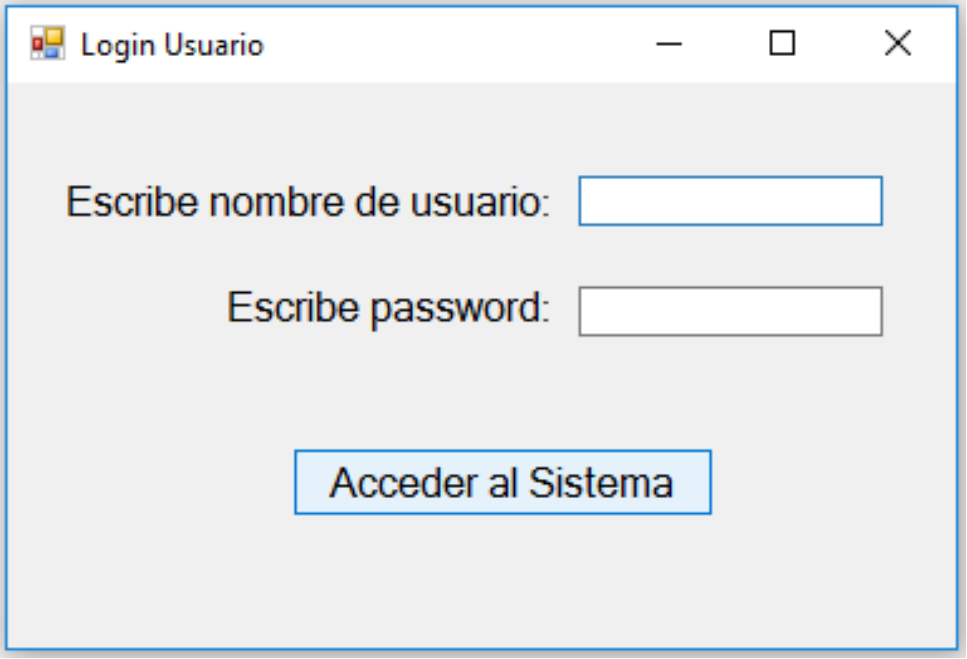
Índice

- Definiciones
- Pruebas de Caja Blanca
- Pruebas de Caja Negra
- Ejemplos prácticos

Definiciones

- **CASO DE PRUEBA**: Es un conjunto de entradas de prueba, condiciones de ejecución y resultados esperados. Tiene un objetivo concreto (probar algo)
- Son una serie de condiciones que se establecen con el objetivo de determinar si la aplicación funciona correctamente según lo esperado.
- **Ejemplo**: CASO de PRUEBA CP1 para CASO de USO “Entrada Sistema”
- **Entrada**: usuario “hacker” password “ejemplo”
- **Condiciones de ejecución**: no existe en la tabla Cuentas (usuario, pass, intentos) la tupla <”hacker”,”ejemplo”, x> pero sí una tupla <”hacker”, “hola”, x>
- **Resultado esperado**: no deja entrar y cambia la tupla a <”hacker”,”hola”,x+1>.
- **Objetivo** del caso de prueba: comprobar que no deja entrar a un usuario existente.

Definiciones



Definiciones

- **PROCEDIMIENTO DE PRUEBA**: Pasos que hay que llevar a cabo para probar uno (o varios) casos de prueba. ¿Cómo probar el caso de prueba y verificar si ha tenido éxito?
- **Ejemplo**: Procedimiento de prueba para Caso de Prueba 1
- Ejecutar la clase Presentación. Comprobar que en la BD “passwords” existe la tupla <”hacker”,”hola”,x>
- Escribir “hacker” en la interfaz gráfica (en el campo de texto etiquetado “Escribe nombre de usuario”. Escribir “ejemplo” en la interfaz gráfica (en el campo de texto “Escribe password”)
- Pulsa el botón “Acceder al sistema”
- Comprobar que no deja entrar al sistema y que en la BD la tupla ha cambiando a <”hacker”,”hola”,x+1>

Definiciones

- Un **buen caso de prueba** es aquel que tiene una probabilidad muy alta de descubrir un nuevo error.
- No debe ser ni muy sencillo ni excesivamente complejo: es mejor realizar cada prueba de forma separada si se quiere probar diferentes casos.
- Una **prueba tiene éxito** si descubre un error. Eso sí, no asegura la ausencia de defectos, sólo puede demostrar que existen errores en el software.
- Para **diseñar los casos de prueba**, vamos a estudiar dos técnicas: **pruebas de caja blanca y pruebas de caja negra.**

Organización de pruebas

- **Prueba Unitarias (de Unidades)**
- Se trata de probar módulos sueltos
- **Fase informal previa**
  - Ejecutar pequeños ejemplos
  - Hay herramientas que analizan automáticamente la sintaxis de los programas (incluso opinando sobre fuentes potenciales de errores)
- **Fase de prueba sistemática**
  - Pruebas de caja blanca (o transparente): Se mira con lupa la estructura del código escrito
  - Pruebas de caja negra: Se prueba la funcionalidad del módulo sin atender a su contenido

Pruebas de Caja Blanca

- Estructurales, Caja Transparente
- **Objetivo:** Probar exhaustivamente la estructura del código
- Aseguran que el **código interno del programa** se ajusta a las especificaciones y que todos los componentes internos se han probado adecuadamente, intentando garantizar que todos los caminos de ejecución del programa quedan probados.
- **Cobertura**: medida del porcentaje de código que ha sido probado o cubierto con las pruebas
  - Cobertura de segmentos
  - Cobertura de ramas
  - Cobertura de condición/decisión
  - Cobertura de bucles

Pruebas de Caja Blanca

- **Cobertura de segmentos (Sentencias)**
- **Segmento**: Secuencia de sentencias sin puntos de decisión.
- El número de sentencias de un programa es finito. Se puede diseñar un plan de pruebas que vaya ejercitando más y más sentencias, hasta que hayamos pasado por todas, o por una inmensa mayoría.
- En la práctica, el proceso de pruebas termina antes de llegar al 100%, pues puede ser excesivamente laborioso y costoso provocar el paso por todas y cada una de las sentencias.
- A la hora de decidir el punto de corte antes de llegar al 100% de cobertura hay que ser precavido y pasar por una mayoría de sentencias elegidas adecuadamente.

Pruebas de Caja Blanca

- **Cobertura de ramas**
- ¿Qué ocurre con los segmentos opcionales?

```
if(condicion) {
    EjecutaEsto ( );
}
```

- Desde el punto de vista de cobertura de segmentos, basta ejecutar una vez, con éxito en la condición, para cubrir todas las sentencias posibles.
- **Hay que probar cuándo la condición se cumple y cuándo falla**
- **Refinamiento de la cobertura de segmentos**: recorrer todas las posibles salidas de los puntos de decisión (y excepcionales)
- Estos criterios se extienden a las construcciones que suponen elegir 1 de entre varias ramas. Por ejemplo, el CASE.

Pruebas de Caja Blanca

- **Cobertura de condición/decisión**
- ¿Qué ocurre cuando la expresión booleana es más compleja?

```
if(condicion1 OR condicion2) {
    EjecutaEsto ( );
}
```

- Sólo 2 ramas, pero 4 posibles combinaciones

Prueba 1: Condicion1 = TRUE y Condicion2 = FALSE
Prueba 2: Condicion1 = FALSE y Condicion2 = TRUE
Prueba 3: Condicion1 = FALSE y Condicion2 = FALSE
Prueba 4: Condicion1 = TRUE y Condicion2 = TRUE

Pruebas de Caja Blanca

- **Cobertura de condición/decisión**
- *Hay que definir un criterio de cobertura sobre las combinaciones*
- Bastan las pruebas 2 y 3 para tener cubiertas todas las ramas. Pero con ellos sólo hemos probado una posibilidad para la Condición1.
- No basta con cubrir cada una de las condiciones componentes, sino que hay que cuidar de sus posibles combinaciones de forma que se logre siempre probar todas y cada una de las ramas.
- El conjunto mínimo de pruebas para cubrir todos los aspectos es el formado por las pruebas 3 y 4. Aún así, nótese que no hemos probado todo lo posible. Por ejemplo, si en el programa nos colamos y ponemos AND donde queríamos poner OR (o viceversa), este conjunto de pruebas no lo detecta. Sólo queremos decir que la cobertura es un criterio útil y práctico; pero no es prueba exhaustiva.

Pruebas de Caja Blanca

- **Cobertura de bucles**
- Los bucles no son más que segmentos controlados por decisiones.
- Así, la cobertura de ramas cubre plenamente la esencia de los bucles. Pero eso es simplemente la teoría, pues la práctica descubre que los bucles son una fuente inagotable de errores, todos triviales, algunos mortales.
- Un bucle se ejecuta un cierto número de veces; pero ese número de veces debe ser muy preciso, y lo más normal es que, ejecutarlo una vez de menos o una vez de más tenga consecuencias indeseables.
- Y, sin embargo, es extremadamente fácil equivocarse y redactar un bucle que se ejecuta 1 vez de más o de menos.

Pruebas de Caja Blanca

- **Cobertura de bucles**
- Para un bucle de tipo **WHILE** hay que pasar 3 pruebas
  - 0 ejecuciones
  - 1 ejecución
  - más de 1 ejecución
- Para un bucle de tipo **REPEAT (DO-WHILE)** hay que pasar 2 pruebas
  - 1 ejecución
  - más de 1 ejecución
- Los bucles **FOR**, en cambio, son muy seguros, pues en su cabecera está definido el número de veces que se va a ejecutar. Basta pues con ejecutarlos 1 vez.

Pruebas de Caja Blanca

- **Cobertura de bucles**
- No obstante, conviene no engañarse con los bucles FOR y examinar su contenido. Si dentro del bucle se altera la variable de control, o el valor de alguna variable que se utilice en el cálculo del incremento o del límite de iteración, entonces eso es un **bucle FOR con trampa**.
- También tiene "trampa" si contiene sentencias del tipo EXIT (que algunos lenguajes denominan **BREAK**) o del tipo RETURN. Todas ellas provocan **terminaciones anticipadas del bucle**.
- Estos últimos párrafos hay que precisarlos para cada lenguaje de programación. Lo peor son aquellos lenguajes que permiten el uso de sentencias **GOTO**.

Pruebas de Caja Blanca

- *Y en la práctica ¿qué hago?*

- Acercarse al 100% de segmentos
- Lograr una buena cobertura de ramas
- La buena cobertura depende del tipo de programa:
  - Juegos basta con probar 60%-80% del código
  - Aplicaciones críticas (sanitarias, centrales nucleares, aplicaciones militares, etc.) > 90%
  - La cobertura requerida suele crecer con el grado de distribución del programa
- Las pruebas de caja blanca se pueden hacer con un depurador (debugger)

Pruebas de Caja Blanca

- ***Limitaciones***
- Lograr una buena cobertura con pruebas de caja blanca es un objetivo deseable; pero no suficiente. Un programa puede estar perfecto, y sin embargo no servir a la función que se pretende.
- Por ejemplo, un Rolls-Royce es un coche que sin duda pasaría las pruebas más exigentes sobre los últimos detalles de su mecánica o su carrocería. Sin embargo, si el cliente desea un todo-terreno, difícilmente va a comprárselo.
- Por ejemplo, si escribimos una rutina para ordenar datos por orden ascendente, pero el cliente los necesita en orden decreciente, no hay prueba de caja blanca capaz de detectar la desviación.
- Las pruebas de caja blanca nos convencen de que un programa hace bien lo que hace; pero no de que haga lo que necesitamos.
- Necesitamos **comprobar la funcionalidad** con las pruebas de **caja negra**

Pruebas de Caja Blanca

```
Public Class Form1
    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
        Dim valor As Integer
        Dim i As Integer

        If TextBox1.Text = "" Then
            MsgBox("Escribe un valor")
        Else
            Try
                valor = CInt(TextBox1.Text)
                If valor < 0 Then
                    MsgBox("Negativo")
                Else
                    For i = 2 To (valor / 2) - 1
                        If (valor Mod i) = 0 Then
                            MsgBox("El número NO es Primo")
                            Exit Sub
                        End If
                    Next
                    MsgBox("El número SÍ es Primo")
                End If
            Catch ex As Exception
                MsgBox("Escribe un valor numérico")
            End Try
        End If
    End Sub
End Class
```

Pruebas de Caja Blanca

Entrada	Resultado	Ejecuciones
	Falta parámetro	TextBox1.text==“” true
-2	Número no positivo	TextBox1.text==0 false valor<=0 true
2	Primo	valor<=0 false Entra if del bucle 0 veces
4	No Primo	valor<=0 false valor mod i==0 Entra if del bucle 1 vez
23	Primo	valor<=0 false Entra if del bucle 0 veces valor mod i=0 false
Palabra	No es un número	Excepción

Pruebas de Caja Negra

- ***Pruebas de caja negra***
- Pruebas de caja opaca, funcionales, entrada/salida e inducidas por los datos. **Objetivo**: Probar la funcionalidad del código
- Se centran en intentar encontrar casos en que el módulo no se atiene a su especificación.
- Las pruebas de caja negra están **especialmente indicadas** en aquellos módulos que van a ser interfaz con el usuario (en sentido general: teclado, pantalla, ficheros, canales de comunicaciones, etc.)
- Las pruebas de caja negra se apoyan en la especificación de requisitos del módulo. De hecho, se habla de **"cobertura de especificación"** para dar una medida del número de requisitos que se han probado. Es fácil obtener coberturas del 100% en módulos internos, aunque puede ser más laborioso en módulos con interfaz al exterior.

Pruebas de Caja Negra

- ***Pruebas de caja negra***

- **Problema**: El conjunto de datos es muy amplio.
- **Solución**: Analizar las clases de equivalencia
  - Esta técnica trata cada parámetro como un modelo algebraico donde unos datos son equivalentes a otros. Si logramos partir un rango excesivamente amplio de posibles valores reales a un conjunto reducido de clases de equivalencia, entonces es suficiente probar un caso de cada clase, pues los demás datos de la misma clase son equivalentes.

Pruebas de Caja Negra

- ***Pruebas de caja negra***

- El problema está pues en identificar clases de equivalencia, ya que no existe una regla de aplicación universal; pero hay consejos para la mayor parte de los casos prácticos:
  - si un parámetro de entrada debe estar comprendido en un cierto rango, aparecen 3 clases de equivalencia: por debajo, en y por encima del rango.
  - si una entrada requiere un valor concreto, aparecen 2 clases de equivalencia: el valor concreto y cualquier valor distinto
  - si una entrada requiere un valor de entre los de un conjunto, aparecen 2 clases de equivalencia: en el conjunto o fuera de él.
  - si una entrada es booleana, hay 2 clases: si o no.
  - los mismos criterios se aplican a las salidas esperadas: hay que intentar generar resultados en todas y cada una de las clases.

Pruebas de Caja Negra

Reglas que ayudan a identificar clases de equivalencia

Condiciones/ Especificaciones de entrada	Clases de entradas VÁLIDAS	Clases de entradas NO VÁLIDAS
Se especifica un RANGO. Ej.: El nº estará comprendido entre 1 y 49	1 clase válida: 1 <=nº <=49	2 clases no válidas: nº <1 nº >49
Especificación de tipo BOOLEANO. Ej.: El primer carácter debe ser una letra	1 clase válida: "es una letra"	1 clase no válida: "no es una letra"
Nº DE VALORES. Ej.: 10 caracteres	1 clase válida: 10 caracteres	2 clases no válidas: < 10 caracteres >10 caracteres
Se especifica un conjunto de VALORES ADMITIDOS. Ej.: Pueden registrarse 3 tipos de inmuebles: pisos, chalés y locales comerciales	1 clase válida por cada valor: piso chalé local comercial	1 clase no válida: Cualquier otro caso. Ej.: garaje

Pruebas de Caja Negra

- Una vez obtenidas las clases de equivalencia, se utilizarán para identificar los casos de prueba correspondientes:
  - Se asigna un nº único a cada clase.
  - Hasta que todas las clases válidas hayan sido cubiertas, se tratará de crear un caso de prueba que cubra tantas clases válidas como sea posible.
  - Hasta que todas las clases no válidas hayan sido cubiertas, se escribirá un caso de prueba por cada clase no válida sin cubrir.

Pruebas de Caja Negra

- **Análisis de valores frontera**
  - Por lo general, los errores ocurren en los límites de las clases equivalentes.
  - Podemos fortalecer nuestras pruebas de caja negra probando los valores que se encuentran en los límites de las clases.
  - En algunos casos, esto significa una prueba adicional.

Especificaciones	**Casos a probar**
RANGO DE VALORES Ej.: Entre 1.0 y 15.0	Límites y valores próximos (no válidos) Probar: 0.9, 1.0 15.0 y 15.1
Nº DE VALORES: Un módulo lee una serie de valores, digamos, 1, 2 o 3 valores	probar con 1 y 3 valores

Pruebas de Caja Negra

- **Limitaciones**
- Lograr una buena cobertura con pruebas de caja negra es un **objetivo deseable**; pero no suficiente a todos los efectos. Un programa puede pasar con holgura millones de pruebas y sin embargo tener defectos internos que surgen en el momento más inoportuno (Ley de Murphy).
- Por ejemplo, un PC que contenga el virus Viernes-13 puede estar pasando pruebas de caja negra durante años y años. Sólo falla si es viernes y es día 13; pero ¿a quién se le iba a ocurrir hacer esa prueba?
- Las pruebas de caja negra nos convencen de que un programa hace lo que queremos; pero no de que haga (además) otras cosas menos aceptables.

Caso Práctico

- **Diseñar casos de prueba para el siguiente procedimiento**
- PUBLIC BOOLEAN Busca (CHAR C, ; CHAR[ ] V)
- El método devuelve TRUE si C está en V, y FALSE en caso contrario

- Primero identificamos las **CLASES DE EQUIVALENCIA**
  - C está poco especificado
  - V no se dice nada de las dimensiones del array ni del criterio de ordenación
  - Lo que devuelve si está claro

- Antes que nada hay que aclarar estos puntos en la especificación del problema

Caso Práctico

- **Diseñar casos de prueba para el siguiente procedimiento**
- Nueva versión de la especificación
- A este procedimiento se le proporciona un carácter C y un array V de caracteres. Se admitirá cualquier carácter de 8 bits de los representables en un PC con Java. El array podrá tener entre 0 y 10.000 caracteres y deberá estar ordenado alfabéticamente, en orden ascendente. Es admisible cualquier cadena de caracteres. El procedimiento devuelve TRUE si C está en V, y FALSE si no. Trabajamos en Java.

Caso Práctico

- **Diseñar casos de prueba para el siguiente procedimiento**
- Ahora podemos identificar las clases de equivalencia
- C: Cualquier carácter
- V:
  - El array vacío
  - Un array entre 1 y 10.000 elementos, ordenado
  - Un array entre 1 y 10.000 elementos, desordenado
- Resultado
  - TRUE
  - FALSE

Caso Práctico

- **Diseñar casos de prueba para el siguiente procedimiento (Clases de equivalencia)**

Cond. Entrada	Clases válidas	Clases no válidas
C (Cualquier carácter)	1. Un carácter válido	2. Cualquier dato que no sea de tipo carácter
V (Array de caracteres de 1 a 10.000 elementos ordenado)	3. 1<= nº elementos array <= 10.000	4. nº elementos < 1 5. nº elementos > 10.000 6. Array desordenado
Resultado	7. TRUE 8. FALSE	9. Cualquier otro resultado

Caso Práctico

- **Diseñar casos de prueba para el siguiente procedimiento (Clases de equivalencia)**

Casos Prueba	Clases Equivalencia	Resultados
‘k’ “bfr”	1,3,8	FALSO
‘k’ “bkr”	1,3,7	TRUE
‘k’ “ “	1,4,8	FALSO
‘k’ 10.001 elementos	1,5,8	FALSO
‘k’ “lcm”	6,9	……
‘NULL’ “bkr”	2,9	……

Caso Práctico

- **Diseñar casos de prueba para el siguiente procedimiento**
- Pruebas de caja negra (ANÁLISIS DE VALORES LÍMITE)

  - Buscar el carácter “k” en el ARRAY “” debe devolver FALSE
  - Buscar el carácter “k” en el ARRAY “k” debe devolver TRUE
  - Buscar el carácter “k” en el ARRAY “j” debe devolver FALSE
  - Buscar el carácter “k” en el ARRAY “kl” debe devolver TRUE
  - Buscar el carácter “k” en el ARRAY “jk” debe devolver TRUE
  - Buscar el carácter “k” en el ARRAY de 10.000 “a” debe devolver FALSE

Caso Práctico

- **Diseñar casos de prueba para el siguiente procedimiento**

```
1.  public boolean Busca ( char C, char V[] ) {
2.    int a, z, m;
3.    a = 0;
4.    z = V.length - 1;
5.    while ( a <= z ) {
6.      m = ( a + z ) / 2;
7.      if ( V[m] == C )
8.        return true;
9.      else {
10.       if ( V[m] < C )
11.         a = m + 1;
12.       else
13.         z = m - 1;
14.     }
15.   }
16.  return false;
17.}
```

Para las pruebas de **caja blanca** necesitamos conocer el código interno

Caso Práctico

- **Diseñar casos de prueba para el siguiente procedimiento**

- Si ejecutamos las pruebas anteriores (Caja negra) marcando por dónde vamos pasando sobre el código, sólo nos queda por probar la **rama de la línea 12**
- Hay que añadir un caso **adicional para** pruebas de caja blanca:
  - Buscar el carácter “k” en el ARRAY “l” debe devolver FALSE
- Cobertura del 100% de segmentos y de condiciones